

Apostila Completa - Stored Procedures SQL Server

Curso de Banco de Dados Relacional

Modulo: Programacao no Banco (Stored Procedures) **SGBD abordado:** SQL Server (T-SQL) **Base pratica usada nesta apostila:** banco 933000081200000173202662_20260127_104141, schemas **dbo** e **fisc** (206 stored procedures reais)

Sumario

1. O que e uma stored procedure
 2. Stored Procedure x Function x Trigger
 3. Sintaxe basica de criacao (CREATE PROCEDURE)
 4. Elementos de uma procedure 4.1 Inspeccionando procedures existentes — consultas de metadados
 5. Parametros de entrada e valores padrao
 6. Formas de passar parametros na chamada
 7. Saidas de uma procedure: OUTPUT, RETURN e result sets
 8. Tabelas temporarias com #
 9. Tabela temporaria x variavel de tabela (@) x tabela permanente
 10. Estudo de caso 1: PR_CADASTRO_CFOP (procedure simples)
 11. Estudo de caso 2: 100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS (parametro com default e controle de fluxo)
 12. Estudo de caso 3: 126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA (multiplos parametros e cadeia de tabelas temporarias)
 13. Boas praticas observadas no banco
 14. Erros comuns
 15. Roteiro de laboratorio
 16. Exercicios propostos
-

1. O que é uma stored procedure

Stored procedure é um objeto de banco que agrupa um ou mais comandos T-SQL sob um nome, compilado e armazenado no banco, para ser executado por chamada explícita (**EXEC**).

Diferente de uma function, uma procedure:

1. Não pode ser chamada dentro de uma expressão de **SELECT** (não se usa **SELECT minha_procedure(x)**).
2. Pode executar **INSERT**, **UPDATE**, **DELETE**, **CREATE TABLE**, controle transacional (**BEGIN TRAN/COMMIT**).
3. Pode devolver zero, um ou vários result sets ao mesmo tempo.
4. Pode receber parâmetros de entrada e devolver parâmetros de saída (**OUTPUT**).

No banco fiscal usado nesta apostila, procedure é a unidade básica de trabalho: cada consulta de auditoria do schema **fisc** (100, 126, 133, 224 ...) é implementada como uma procedure em **dbo** que materializa o resultado em tabela e/ou devolve result sets.

2. Stored Procedure x Function x Trigger

Na prática fiscal, as três construções resolvem problemas diferentes. O critério mais importante não é “qual é mais poderosa”, e sim: “qual resposta a rotina precisa dar?”

- A **stored procedure** executa um processo: gera relatório, recalcula painel, atualiza tabela, materializa resultado.
- A **function** responde uma pergunta: “qual é o valor do imposto?”, “o documento está inconsistente?”, “qual foi a última nota do contribuinte?”.
- O **trigger** reage automaticamente ao evento: “um registro foi inserido, alterado ou excluído; vou validar, bloquear ou registrar isso”.

2.1 Comparação rápida no contexto fiscal

Recurso	Retorno	Como é chamada	Melhor uso em auditoria tributária
Stored Procedure	SELECT, OUTPUT, RETURN, múltiplos result sets	EXEC nome_procedure	Reprocessar painel fiscal, gerar relatório, materializar tabelas intermediárias, ETL de auditoria
Function	Escalar ou tabela	Dentro de SELECT, WHERE, JOIN ou expressão	Calcular imposto, validar regra, transformar código em descrição, reutilizar lógica de consulta
Trigger	Sem retorno direto	Disparada automaticamente em INSERT/UPDATE/DELETE	Validar consistência, registrar log, impedir gravação indevida, manter integridade de processo fiscal

2.2 Exemplo fiscal: stored procedure

Uma **stored procedure** é a escolha correta quando o objetivo é executar uma rotina de negócio completa, como “recalcular todas as divergências da EFD e materializar o resultado para análise”.

```
CREATE PROCEDURE dbo.PR_GERAR_DIVERGENCIA_NFE_EFD
    @ANO INT
AS
BEGIN
    SET NOCOUNT ON;

    DROP TABLE IF EXISTS #divergencias;

    SELECT
        n.id_nfe,
        n.chave_acesso,
        n.valor_total,
        e.valor_documento,
        n.valor_total - e.valor_documento AS diferenca_valor
    INTO #divergencias
    FROM dbo.nfe AS n
    LEFT JOIN dbo.efd_c100 AS e
```

```

        ON e.chave_acesso = n.chave_acesso
WHERE YEAR(n.data_emissao) = @ANO;

SELECT *
FROM #divergencias
WHERE ABS(diferenca_valor) > 0;
END;
GO

```

Quando usar:

- gerar painel de auditoria;
- recalcular indicadores fiscais;
- montar tabela intermediária para consulta posterior;
- atualizar dados em lote.

Melhor uso: processo completo, com passos, filtros, temporárias e saída final organizada.

2.3 Exemplo fiscal: **function**

Uma **function** serve para responder uma pergunta dentro de outra consulta. Em auditoria fiscal, ela costuma encapsular uma regra sempre repetida.

```

CREATE FUNCTION fisc.fn_status_documento(@chave_acesso CHAR(44))
RETURNS VARCHAR(30)
AS
BEGIN
    DECLARE @status VARCHAR(30);

    IF EXISTS (
        SELECT 1
        FROM dbo.nfe
        WHERE chave_acesso = @chave_acesso
        AND situacao = 'CANCELADA'
    )
        SET @status = 'CANCELADA';
    ELSE IF EXISTS (
        SELECT 1
        FROM dbo.nfe
        WHERE chave_acesso = @chave_acesso
    )
        SET @status = 'AUTORIZADA';
    ELSE
        SET @status = 'NAO_ENCONTRADA';

    RETURN @status;

```

```
END;  
GO
```

Uso na consulta:

```
SELECT  
    e.id_c100,  
    e.chave_acesso,  
    fisc.fn_status_documento(e.chave_acesso) AS status_nfe  
FROM dbo.efd_c100 AS e  
WHERE e.periodo_apuracao = '202501';
```

Quando usar:

- transformar um código em texto legível;
- validar regra de negócio em uma consulta;
- reaproveitar lógica sem depender de procedimento procedural.

Melhor uso: resposta calculada dentro de **SELECT**, **WHERE**, **JOIN** ou expressão.

Atenção: em T-SQL, **function** não faz **INSERT**, **UPDATE**, **DELETE** e não é a melhor ferramenta para processar grande volume de dados em lote. Ela existe para cálculo e reutilização logicamente embutida na consulta.

2.4 Exemplo fiscal: **trigger**

O **trigger** é para reagir automaticamente quando um dado muda. Em ambiente fiscal, esse tipo de comportamento é útil para evitar inconsistência no cadastro ou registrar evento de auditoria.

```
CREATE TRIGGER dbo.trg_validar_nota_duplicada  
ON dbo.nfe  
AFTER INSERT, UPDATE  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    IF EXISTS (  
        SELECT 1  
        FROM dbo.nfe AS n  
        JOIN inserted AS i  
            ON i.chave_acesso = n.chave_acesso  
        WHERE n.id_nfe <> i.id_nfe  
    )  
    BEGIN
```

```
RAISERROR('Chave de acesso duplicada. Documento fiscal rejeitado.', 16, 1);  
ROLLBACK TRANSACTION;  
END;  
END;  
GO
```

Quando usar:

- bloquear cadastro duplicado de NF-e;
- impedir gravação de CNPJ com formato inválido;
- gravar log de auditoria ao inserir ou alterar registros críticos;
- disparar validações automatizadas quando o dado entra no sistema.

Melhor uso: resposta automática a um evento DML, sem chamada explícita do usuário.

2.5 Resumo prático para explicar em aula

1. **Stored Procedure** = executa um processo de auditoria ou ETL.

- Exemplo: recalcular divergências de EFD/NF-e, criar relatório de inconsistências, gerar painel de fiscalização.

2. **Function** = responde uma pergunta em contexto de consulta.

- Exemplo: “qual é o status da nota?”, “qual é o valor total do imposto?”, “a operação é regular ou cancelada?”.

3. **Trigger** = reage automaticamente a alterações no dado.

- Exemplo: impedir duplicidade de chave de acesso, registrar auditoria, rejeitar gravação inconsistente.

2.6 Regra prática da equipe

- Use **stored procedure** quando tiver um fluxo de trabalho ou rotina operacional.
- Use **function** quando a lógica é uma expressão/consulta reutilizável.
- Use **trigger** apenas para garantir integridade/reação automática a eventos de dados.

Em auditoria fiscal, a regra mais importante é esta: o processo de análise pertence à **stored procedure**; a regra de cálculo ou classificação pertence à

3. Sintaxe basica de criacao (CREATE PROCEDURE)

```
CREATE PROCEDURE [schema].[nome_da_procedure]
    @parametro1 TIPO,
    @parametro2 TIPO = valor_padrao
AS
SET NOCOUNT ON;
BEGIN
    -- corpo da procedure
END
GO
```

Pontos de sintaxe:

- **CREATE PROCEDURE** pode ser abreviado para **CREATE PROC**.
- **CREATE OR ALTER PROCEDURE** cria se nao existir e substitui se ja existir (evita ter que fazer **DROP** manual antes de recriar).
- O **GO** nao e um comando T-SQL, e um separador de lote reconhecido pelo **sqlcmd/SSMS**; ele fecha o **CREATE PROCEDURE**, que precisa ser o unico comando do lote.
- **SET NOCOUNT ON** evita que o SQL Server devolva a mensagem (**n rows affected**) apos cada comando -- reduz trafego de rede e evita que ferramentas clientes interpretem essa mensagem como um result set extra. E convencao em praticamente toda procedure de producao.

Exemplo minimo, no padrao deste banco:

```
CREATE PROCEDURE [DBO].[PR_CADASTRO_CFOP]
AS
SET NOCOUNT ON;
BEGIN
    SELECT CDCFOP, NATUREZA, DESCRICAO, COMPLEMENTO
    FROM BI_DW_CORPORATIVO_FISC.DBO.TBCAD_CFOP
    ORDER BY CDCFOP;
END
```

4. Elementos de uma procedure

Elemento	Funcao
<code>CREATE PROCEDURE</code> <code>[schema].[nome]</code>	Cabecalho: onde a procedure fica e como se chama
Lista de parametros	Entradas (e, opcionalmente, saidas) da procedure
<code>AS</code>	Separa o cabecalho do corpo
<code>SET NOCOUNT ON</code>	Configuracao de sessao, primeira linha do corpo por convencao
<code>BEGIN ... END</code>	Delimita o bloco principal (opcional para um unico comando, mas usado por convencao em todas as procedures deste banco)
Corpo (DML, DDL, controle de fluxo)	A logica em si: <code>SELECT</code> , <code>INSERT</code> , <code>IF</code> , <code>WHILE</code> , tabelas temporarias etc.
<code>RETURN</code> (opcional)	Encerra a execucao e pode devolver um inteiro de status
<code>GO</code>	Fecha o lote de criacao (nao faz parte da procedure)

4.1 Inspeccionando procedures existentes — consultas de metadados

Antes de criar ou alterar uma procedure, e essencial saber o que ja existe no banco — quais procedures estao no schema, quando foram modificadas e quais parametros cada uma aceita. O SQL Server expoe essas informacoes em visoes de catalogo (`sys.*`) e na visao padrao ANSI `INFORMATION_SCHEMA.ROUTINES`.

Listar todas as procedures de um banco ou schema

```
-- Via sys.procedures: mais completo, inclui data de criacao/alteracao e flag de  
-- schemabinding  
SELECT  
    SCHEMA_NAME(schema_id) AS schema_nome,
```



```
name AS procedure_nome,  
create_date,  
modify_date  
FROM sys.procedures  
ORDER BY schema_nome, procedure_nome;
```

Filtrado por schema especifico (ex.: listar apenas as procedures do schema **dbo** neste banco):

```
SELECT  
    SCHEMA_NAME(schema_id) AS schema_nome,  
    name AS procedure_nome,  
    create_date,  
    modify_date  
FROM sys.procedures  
WHERE SCHEMA_NAME(schema_id) = 'dbo'  
ORDER BY name;
```

Buscar pelo padrao de nomenclatura usado neste banco (**_PR_**):

```
SELECT name AS procedure_nome, create_date, modify_date  
FROM sys.procedures  
WHERE name LIKE '%PR%'  
ORDER BY name;
```

Via **INFORMATION_SCHEMA.ROUTINES** (padrao ANSI)

```
SELECT  
    ROUTINE_SCHEMA AS schema_nome,  
    ROUTINE_NAME AS procedure_nome,  
    CREATED,  
    LAST_ALTERED  
FROM INFORMATION_SCHEMA.ROUTINES  
WHERE ROUTINE_TYPE = 'PROCEDURE'  
    AND ROUTINE_SCHEMA = 'dbo'  
ORDER BY ROUTINE_NAME;
```

INFORMATION_SCHEMA.ROUTINES e portavel entre SGBDs, mas no SQL Server nao exibe procedures de sistema nem as do schema **sys**. Para inspecao completa dentro do SQL Server, prefira **sys.procedures**.

Ver os parametros de uma procedure especifica

```
-- Lista todos os parametros de uma procedure com nome, tipo e se e OUTPUT
SELECT
    p.name                                AS parametro,
    TYPE_NAME(p.user_type_id)            AS tipo,
    p.max_length,
    p.is_output,
    p.has_default_value,
    p.default_value
FROM sys.parameters AS p
WHERE OBJECT_NAME(p.object_id) =
'126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA'
ORDER BY p.parameter_id;
```

Para confirmar (como visto no roteiro de laboratorio da secao 15) que **nenhuma procedure deste banco usa parametro OUTPUT**:

```
SELECT
    OBJECT_NAME(object_id) AS procedure_nome,
    name                    AS parametro
FROM sys.parameters
WHERE is_output = 1;
-- resultado vazio neste banco: nenhuma procedure usa OUTPUT
```

Ver o codigo-fonte de uma procedure

```
-- Opcao 1: retorna o codigo como uma unica string
SELECT OBJECT_DEFINITION(OBJECT_ID('dbo.PR_CADASTRO_CFOP'));

-- Opcao 2: retorna o codigo linha a linha (mais legivel no SSMS)
EXEC sp_helptext 'dbo.PR_CADASTRO_CFOP';
```

Buscar o codigo-fonte de todas as procedures que referenciam uma tabela ou coluna

Util para entender impacto antes de alterar uma tabela usada por procedures de auditoria:

```
SELECT
    SCHEMA_NAME(p.schema_id)      AS schema_nome,
    p.name                        AS procedure_nome,
    m.definition                  AS codigo_sql
FROM sys.procedures AS p
JOIN sys.sql_modules AS m ON m.object_id = p.object_id
WHERE m.definition LIKE '%EFD_C190%' -- substitua pelo nome da tabela/coluna
buscada
ORDER BY p.name;
```

Identificar dependencias: quais objetos uma procedure referencia

```
SELECT
    referenced_schema_name AS dep_schema,
    referenced_entity_name AS dep_objeto,
    referenced_minor_name  AS dep_coluna
FROM sys.dm_sql_referenced_entities(
    'dbo.126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA',
    'OBJECT'
)
ORDER BY referenced_entity_name;
```

Tabela-resumo: quando usar cada consulta de metadados

Objetivo	Consulta recomendada
Listar todas as procedures do banco	<code>SELECT ... FROM sys.procedures</code>
Filtrar procedures por schema ou nome	<code>sys.procedures WHERE SCHEMA_NAME(...) = 'dbo' e/ou name LIKE '%PR%'</code>
Ver parametros e tipos de uma procedure	<code>sys.parameters WHERE OBJECT_NAME(object_id) = '...'</code>
Confirmar ausencia de parametros OUTPUT	<code>sys.parameters WHERE is_output = 1</code>

Objetivo	Consulta recomendada
Ver o código-fonte de uma procedure	<code>OBJECT_DEFINITION(OBJECT_ID(...))</code> ou <code>sp_helptext</code>
Encontrar procedures que usam uma tabela	<code>sys.sql_modules WHERE definition LIKE '%nome_tabela%'</code>
Analisar dependências de objetos	<code>sys.dm_sql_referenced_entities(...)</code>

5. Parametros de entrada e valores padrao

Parametros sao declarados logo apos o nome da procedure, com `@nome TIPO`. Podem ter valor padrao com `= valor`, o que os torna opcionais na chamada.

Exemplo real

(`126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA`):

```
CREATE PROCEDURE [DBO].  
[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]  
    @ANO INTEGER = NULL,  
    @PERIODO_DECLARACAO CHAR(7) = NULL,  
    @RELATORIO INTEGER = NULL  
AS
```

Os tres parametros tem `= NULL` como padrao -- ou seja, todos sao opcionais. Isso permite dois modos de uso da mesma procedure:

- `EXEC ...126_PR... @RELATORIO = 99999; -- gera (recalcula) o resultado.`
- `EXEC ...126_PR... @ANO = 2021; -- apenas le o resultado ja gerado, filtrado por ano.`

Esse padrao -- um parametro "modo de operacao" (`@RELATORIO`) que decide se a procedure recalcula do zero ou so consulta o que ja foi calculado -- se repete em varias procedures do schema `fisc` deste banco (`100_PR...`, `126_PR...` e outras da familia `_PR_`).

6. Formas de passar parametros na chamada

SQL Server aceita duas formas, que podem ate ser misturadas (com restricao de ordem):

6.1 Passagem posicional

```
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA] 2021,
'05/2021', NULL;
```

Os valores sao atribuidos na ordem exata da declaracao (@ANO, depois @PERIODO_DECLARACAO, depois @RELATORIO). Funciona, mas e fragil: se a ordem dos parametros mudar na procedure, a chamada quebra silenciosamente (ou pior, passa o valor para o parametro errado sem erro).

6.2 Passagem nomeada (recomendada)

```
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]
@PERIODO_DECLARACAO = '05/2021',
@ANO = 2021;
```

Com @nome = valor, a ordem deixa de importar e parametros com valor padrao podem ser omitidos livremente. E a forma usada em todos os exemplos desta apostila.

6.3 Omitindo parametros opcionais

```
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS]; -- usa
@RELATORIO = NULL (padrao)
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS] @RELATORIO = 99999; --
forca recalculo
```

7. Saidas de uma procedure: OUTPUT, RETURN e result sets

SQL Server oferece tres mecanismos de saida, e vale entender os tres mesmo quando o banco em analise so usa um deles.

7.1 Parametro OUTPUT (nao usado neste banco)

```
CREATE PROCEDURE dbo.exemplo_output
    @id_contribuinte INT,
    @total_notas INT OUTPUT
AS
BEGIN
    SELECT @total_notas = COUNT(*)
    FROM dbo.NFE
    WHERE sqnfe = @id_contribuinte;
END
```

Chamada:

```
DECLARE @qtd INT;
EXEC dbo.exemplo_output @id_contribuinte = 10, @total_notas = @qtd OUTPUT;
SELECT @qtd;
```

O parametro precisa ser declarado com **OUTPUT** tanto na criacao da procedure quanto na chamada. E o mecanismo indicado quando se quer devolver um unico valor para ser usado em outro trecho de codigo (outra procedure, uma variavel, um IF).

Levantamento real neste banco: consultando `sys.parameters` nas 206 procedures existentes, nenhuma usa parametro **OUTPUT**. Todas devolvem dado por result set (**SELECT**) ou por tabela materializada -- o item 7.3 explica por que essa e a escolha certa para o caso de uso delas.

7.2 RETURN (codigo de status)

```
IF @PERIODO_DECLARACAO IS NULL
BEGIN
    RETURN -1; -- codigo de erro
END
```

RETURN encerra a procedure imediatamente e pode devolver um inteiro (nao serve para devolver texto, decimal ou result set). Uso tipico: codigo de sucesso/erro que quem chamou testa com **EXEC @status = dbo.procedure**

7.3 Result sets -- a saida usada neste banco

A forma mais simples de "saida" de uma procedure e simplesmente executar um **SELECT** sem redirecionar o resultado para variavel nenhuma -- ele e devolvido como um result set para quem chamou (SSMS, aplicacao, **sqlcmd**). E possivel devolver **varios** result sets numa mesma execucao, um por **SELECT** solto no corpo.

Exemplo real (**100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS**, trecho final):

```
IF @RELATORIO IS NULL
BEGIN
    IF EXISTS (SELECT NAME FROM SYS.OBJECTS WHERE OBJECT_ID =
OBJECT_ID(N'FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01') AND TYPE IN
(N'U'))

        SELECT NUMERO_CONSULTA, DESCRICAO_CONSULTA, INFOR_DADOS
        FROM FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01
        ORDER BY NUMERO_CONSULTA
END;
```

A procedure **126_PR_...** leva essa ideia adiante: quando **@RELATORIO IS NULL**, ela devolve **ate quatro result sets em sequencia** (um por tabela **TB_126_..._01** a **_04**), cada um filtrado pelos parametros **@ANO/@PERIODO_DECLARACAO**.

Por que esse projeto faz sentido aqui: as procedures deste banco alimentam paineis e planilhas de auditoria consumidos por ferramentas de BI e por analistas via SSMS -- essas ferramentas leem result sets nativamente, enquanto consumir um parametro **OUTPUT** exigiria escrever codigo cliente especifico. Result set tambem permite devolver **multiplas linhas e colunas**, algo que **OUTPUT** (um valor por parametro) nao faz.

8. Tabelas temporarias com

Tabela temporaria e uma tabela criada durante a execucao, armazenada fisicamente em **tempdb**, mas referenciada pelo nome comum como se fosse uma tabela do banco atual.

8.1 Tabela temporaria local (#nome)

```
DROP TABLE IF EXISTS #126_C190_ST;

SELECT N.SQNFOUTRA, NRCFOP,
       SUM(CASE WHEN VLICMS IS NULL THEN 0 ELSE VLICMS END) AS VLICMS
INTO #126_C190_ST
FROM DBO.EFD_C190 N
INNER JOIN BI_DW_CORPORATIVO_FISC.DBO.TBLEG_CFOP O
      ON N.SQCFOP = O.SQCFOP AND NRCFOP IN
(1403,1407,1409,1411,1415,2403,2407,2409,2411,2415)
GROUP BY N.SQNFOUTRA, NRCFOP
HAVING SUM(CASE WHEN VLICMS IS NULL THEN 0 ELSE VLICMS END) > 0;
```

Caracteristicas de uma tabela #:

- Visivel apenas na **sessao** que a criou (e em procedures chamadas por essa sessao).
- E automaticamente descartada quando a sessao termina, ou quando a procedure que a criou termina (se foi criada dentro de uma procedure e nao existia antes dela).
- Pode ter indice, constraint, estatisticas -- e uma tabela de verdade, so que de vida curta.
- **SELECT ... INTO #tabela** e a forma mais comum de cria-la: cria a tabela com as colunas e tipos do resultado da consulta e ja insere as linhas, em um unico comando.

8.2 Tabela temporaria global (##nome)

```
SELECT * INTO ##temp_global FROM dbo.NFE;
```


Visível para **todas as sessões** conectadas ao servidor, e só é descartada quando a sessão que a criou termina e nenhuma outra sessão está mais referenciando ela. Uso raro em código de produção (risco de colisão de nome entre sessões concorrentes); nenhuma procedure deste banco usa **##**.

8.3 O padrão **DROP TABLE IF EXISTS #tabela** antes de cada **SELECT INTO**

Esse padrão aparece dezenas de vezes na procedure **126_PR_...**. Ele existe porque **SELECT ... INTO #tabela** falha com erro se a tabela **#tabela** já existir -- e, como o SSMS costuma reaproveitar a mesma conexão/sessão entre execuções, uma tabela **#** criada numa execução anterior (que travou no meio, por exemplo) pode sobrar na sessão. O **DROP TABLE IF EXISTS** antes de cada **SELECT INTO** torna a procedure segura para ser executada várias vezes seguidas na mesma sessão sem erro.

9. Tabela temporária x variável de tabela (@) x tabela permanente

Característica	Tabela #temp	Variável @tabela	Tabela permanente
Escopo	Sessão (ou procedure)	Batch/procedure atual	Banco de dados
Onde fica	tempdb	tempdb (internamente)	Arquivo de dados do banco
Aceita índice após criação	Sim	Limitado	Sim
Estatísticas de otimização	Sim	Não (pre-SQL Server 2019 sem OPTION(RECOMPILE))	Sim
Pode ser usada em SELECT INTO	Sim	Não (precisa DECLARE ... TABLE)	Sim

Característica	Tabela#temp	Variavel@tabela	Tabela permanente
Uso neste banco	Extensivo em 126_PR_... (calculo intermediario, multi-etapas)	Nao encontrado nas procedures examinadas	100_PR_... grava direto em FISC.TB_100_..._01, uma tabela permanente, em vez de usar #temp

O ponto mais importante para quem esta aprendendo: **a escolha entre #temp e tabela permanente e uma decisao de projeto**, nao uma obrigacao da linguagem. A procedure 100_PR_... grava o resultado final numa tabela permanente (FISC.TB_100_..._01) porque esse resultado precisa **persistir** entre chamadas (o modo @RELATORIO IS NULL so funciona porque a tabela continua existindo depois que a procedure termina). Ja a procedure 126_PR_... usa #temp para as **etapas intermediarias** de calculo (que nao precisam sobreviver ao fim da execucao) e so materializa em tabela permanente (FISC.TB_126_..._01 a _04) o resultado final.

10. Estudo de caso 1: PR_CADASTRO_CFOP (procedure simples)

O que ela faz: devolve a tabela de referencia de CFOP (codigo, natureza, descricao, complemento) usada por outras procedures de auditoria fiscal para "traduzir" o codigo numerico do CFOP em texto legivel.

```
CREATE PROCEDURE [DBO].[PR_CADASTRO_CFOP]
AS
SET NOCOUNT ON;
BEGIN

PRINT 'O TUTORIAL DESTA CONSULTA ENCONTRA-SE NO ROTEIRO OPERACIONAL DO BDFISC.
FAVOR, CONSULTA-LO.';

DROP TABLE IF EXISTS #CADCFOP;

SELECT CDCFOP,NATUREZA,DESCRICAO,COMPLEMENTO
INTO #CADCFOP
FROM BI_DW_CORPORATIVO_FISC.DBO.TBCAD_CFOP;

IF EXISTS (SELECT CDCFOP FROM #CADCFOP)

SELECT
```

```

CASE WHEN CDCFOP IS NULL THEN '-' ELSE CDCFOP END AS CFOP,
CASE WHEN NATUREZA IS NULL THEN '-' ELSE NATUREZA END AS NATUREZA,
CASE WHEN DESCRICAO IS NULL THEN '-' ELSE DESCRICAO END AS DESCRICAO,
CASE WHEN COMPLEMENTO IS NULL THEN '-' ELSE COMPLEMENTO END AS COMPLEMENTO
FROM #CADCDFOP
ORDER BY CDCFOP;

ELSE
SELECT 'SEM DADOS PARA EXIBICAO' AS ALERTA;

DROP TABLE IF EXISTS #CADCDFOP;

END

```

Visao geral, elemento por elemento:

1. **Sem parametros** -- e a procedure mais simples do estudo de caso: nao recebe entrada nenhuma, sempre devolve a tabela inteira.
2. **PRINT** manda uma mensagem informativa para a aba de mensagens do SSMS (nao e um result set, so texto de log).
3. **Uma unica tabela temporaria (#CADCDFOP)**, criada com **SELECT ... INTO** a partir de uma tabela de outro banco (**BI_DW_CORPORATIVO_FISC**, catalogo corporativo da SEFAZ).
4. **IF EXISTS (...) SELECT ... ELSE SELECT ...** -- padrao de "saida com fallback": se a tabela temporaria tiver linhas, devolve o cadastro formatado; se nao tiver (por exemplo, se o banco corporativo estiver indisponivel), devolve uma unica linha de alerta em vez de um result set vazio silencioso. Esse padrao aparece em praticamente todas as procedures **_PR_** deste schema.
5. **DROP TABLE IF EXISTS #CADCDFOP** no fim -- limpeza explicita, embora a tabela **#** ja fosse descartada automaticamente ao fim da procedure; e uma pratica defensiva comum quando a mesma sessao pode chamar a procedure de novo em seguida.

Neste ambiente de laboratorio, **PR_CADASTRO_CFOP** falha ao executar porque **BI_DW_CORPORATIVO_FISC** (o banco corporativo vinculado) nao existe localmente -- o mesmo motivo que impede a **126_PR_...** de recalculer seu resultado aqui (ver [roteiro-auditores-fisc-cfop-indevido-substituicao-tributaria.md](#)).

Execucao de exemplo

```
EXEC dbo.[PR_CADASTRO_CFOP];
```

Como a procedure nao recebe parametros, a chamada e direta: ela gera a tabela **#CADCFOF** e devolve o cadastro de CFOP formatado. Em uma sessao do SSMS, o resultado aparece como um result set final. A estrutura e simples, mas vale como estudo de caso para mostrar o padrao de **SELECT ... INTO #tabela + IF EXISTS + SELECT final**.

Sugestao de reescrita usando visao

Se a ideia for apenas consultar o cadastro sem materializar o resultado em tabela temporaria, a versao mais simples pode ser uma view:

```
CREATE VIEW dbo.vw_cadastro_cfop AS
SELECT
    CASE WHEN CDCFOP IS NULL THEN '-' ELSE CAST(CDCFOP AS VARCHAR(10)) END AS CFOP,
    CASE WHEN NATUREZA IS NULL THEN '-' ELSE NATUREZA END AS NATUREZA,
    CASE WHEN DESCRICAO IS NULL THEN '-' ELSE DESCRICAO END AS DESCRICAO,
    CASE WHEN COMPLEMENTO IS NULL THEN '-' ELSE COMPLEMENTO END AS COMPLEMENTO
FROM BI_DW_CORPORATIVO_FISC.DBO.TBCAD_CFOP;
GO

SELECT *
FROM dbo.vw_cadastro_cfop
ORDER BY CFOP;
```

A vantagem da view e a simplicidade de consumo. A desvantagem e que ela nao substitui a procedure quando o processo exige validacao de parametro, materializacao intermediaria ou **IF** de fluxo operacional.

11. Estudo de caso 2:

100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS (parametro com default e controle de fluxo)

O que ela faz: varre cerca de 34 tabelas de resultado de outras procedures de auditoria (**FISC.TB_101_...**, **FISC.TB_111_...** etc.) e monta um painel de uma linha por consulta, indicando se aquela consulta encontrou ou nao dado para analise. E o "painel de triagem" do auditor -- ja documentado em [roteiro-auditores-fisc-sumario-inconsistencias.md](#).

```

CREATE PROCEDURE [dbo].[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS]
@RELATORIO INTEGER = NULL
AS
SET NOCOUNT ON;
BEGIN

IF @RELATORIO = 99999
BEGIN

DECLARE @QTDE_REG_TB1 INTEGER = 0
DECLARE @QTDE_REG_TB2 INTEGER = 0

DROP TABLE IF EXISTS FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01;

CREATE TABLE FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01 (
NUMERO_CONSULTA INTEGER NOT NULL,
DESCRICAO_CONSULTA VARCHAR (100) NOT NULL,
INFOR_DADOS NVARCHAR(40) NOT NULL
);

IF EXISTS (SELECT NAME FROM SYS.OBJECTS WHERE OBJECT_ID =
OBJECT_ID(N'FISC.TB_116_PR_ENTRADAS_CREDITOS_SAIDAS_NAO_TRIBUTADAS_ITENS_PARA_ANALI
SE') AND
TYPE IN (N'U'))
BEGIN
SET @QTDE_REG_TB1 = (SELECT COUNT(*) FROM
FISC.TB_116_PR_ENTRADAS_CREDITOS_SAIDAS_NAO_TRIBUTADAS_ITENS_PARA_ANALISE)
IF @QTDE_REG_TB1 = 0
INSERT INTO FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01
SELECT 116,
'116_PR_ENTRADAS_CREDITOS_SAIDAS_NAO_TRIBUTADAS_ITENS_PARA_ANALISE', 'CONSULTA NAO
CONTEM DADOS PARA ANALISE';
ELSE
INSERT INTO FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01
SELECT 116,
'116_PR_ENTRADAS_CREDITOS_SAIDAS_NAO_TRIBUTADAS_ITENS_PARA_ANALISE', 'CONSULTA
CONTEM DADOS PARA ANALISE';
END
-- (mesmo padrao se repete para cada uma das ~34 tabelas PR_1xx)
END

IF @RELATORIO IS NULL
BEGIN
IF EXISTS (SELECT NAME FROM SYS.OBJECTS WHERE OBJECT_ID =
OBJECT_ID(N'FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01') AND TYPE IN
(N'U'))
SELECT NUMERO_CONSULTA,DESCRICAO_CONSULTA,INFOR_DADOS
FROM FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01
ORDER BY NUMERO_CONSULTA
END;

END
GO

```

Visao geral, elemento por elemento:

1. **Um parametro, @RELATORIO INTEGER = NULL** -- controla o "modo" da procedure. = 99999 recalcula o painel do zero; deixar de fora (usa o NULL padrao) so le o painel ja calculado.
2. **Variaveis locais (DECLARE @QTDE_REG_TB1 INTEGER = 0)** -- diferente de parametro, uma variavel DECLARED existe so dentro da execucao atual e nao pode ser passada na chamada; serve para guardar resultados intermediarios (aqui, a contagem de linhas de cada tabela PR_1xx) e reaproveitar em IFs seguintes.
3. **Tabela permanente, nao temporaria** -- repare que a procedure usa DROP TABLE IF EXISTS FISC.TB_100_..._01 seguido de CREATE TABLE (sem #). E proposital: o resultado precisa sobreviver ao fim da procedure, porque o modo @RELATORIO IS NULL depende de essa tabela ainda existir na proxima chamada.
4. **Cadeia de IF EXISTS (...) ... SET ... IF ... INSERT ... ELSE INSERT ...** -- o mesmo bloco de 6 linhas se repete ~34 vezes, uma por consulta de auditoria monitorada; cada bloco testa se a tabela de resultado daquela consulta existe, conta as linhas, e grava no painel se "CONTEM DADOS" ou "NAO CONTEM DADOS".
5. **Saida por result set condicional** -- so devolve o SELECT final quando @RELATORIO IS NULL (modo leitura); no modo recalculo (@RELATORIO = 99999) a procedure so grava a tabela, sem devolver result set.

Exemplos de execucao com parametros

```
-- Modo leitura: apenas consulta o painel ja materializado
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS];

-- Modo recalculo: gera ou atualiza a tabela FISC.TB_100_..._01
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS]
    @RELATORIO = 99999;
```

A primeira chamada instala a ideia de consumo simples: a procedure atua como um “painel” de auditoria. A segunda chamada e o modo operacional: ela recalcula todo o conjunto de indicadores e grava na tabela permanente, pronta para ser consultada depois.

Sugestao de reescrita usando visao

Para evitar a tabela materializada, a logica final pode ser convertida em uma view que devolve o mesmo painel sem o passo de persistencia:

```
CREATE VIEW fisc.vw_sumario_consultas_inconsistencias AS
SELECT
    NUMERO_CONSULTA,
    DESCRICAO_CONSULTA,
    INFOR_DADOS
FROM FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01;
GO

SELECT *
FROM fisc.vw_sumario_consultas_inconsistencias
ORDER BY NUMERO_CONSULTA;
```

A vantagem e a facilidade de consulta direta. A limitacao e clara: view nao substitui a rotina de recalculo; ela so expoe o resultado final ja gerado. Em outras palavras, para “processar e persistir” a procedure continua sendo a melhor escolha.

12. Estudo de caso 3:

126_PR_ENTRADAS_CREDITO_C190_CFOP_S UBST_TRIBUT_MERCADORIAS_REVENDA (multiplos parametros e cadeia de tabelas temporarias)

O que ela faz: identifica entradas em que a empresa tomou credito proprio de ICMS sob um CFOP que deveria estar coberto por Substituicao Tributaria (mercadoria de revenda) -- documentada em detalhe em [roteiro-auditores-fisc-cfop-indevido-substituicao-tributaria.md](#). E a procedure mais complexa das tres (mais de 600 linhas), e serve de estudo de caso para tabelas temporarias em cadeia.

```
CREATE PROCEDURE [DBO].
[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]
@ANO INTEGER = NULL,
@PERIODO_DECLARACAO CHAR(7) = NULL,
@RELATORIO INTEGER = NULL
AS
SET NOCOUNT ON;
BEGIN
```

```

DROP TABLE IF EXISTS #126_C190_ST_ITEM;
DROP TABLE IF EXISTS #126_PRIM_PLANCAMENTO;
DROP TABLE IF EXISTS #126_C190_ST;
-- ... (mais 12 DROP TABLE IF EXISTS, um por tabela temporaria usada na procedure)

-- 1) validacao dos parametros de periodo/ano (normaliza formato "5/2021" para
"05/2021" etc.)
SET @PERIODO_DECLARACAO =
CASE WHEN LEN(@PERIODO_DECLARACAO) = 7 AND
CONVERT(INTEGER,SUBSTRING(@PERIODO_DECLARACAO,1,2)) BETWEEN 1 AND 9
THEN SUBSTRING(@PERIODO_DECLARACAO,2,6) ELSE @PERIODO_DECLARACAO END;

IF ((SUBSTRING(@PERIODO_DECLARACAO, 2,1) = '/' AND @ANO <>
SUBSTRING(@PERIODO_DECLARACAO, 3,4)) OR ...)
AND @PERIODO_DECLARACAO IS NOT NULL AND @ANO IS NOT NULL
SELECT 'ERRO: ANO E PERIODO DIVERGENTES. O PERIODO DEVE ESTAR CONTIDO NO ANO' AS
ALERTA;
ELSE
BEGIN

IF @RELATORIO = 99999
BEGIN

-- 2) primeira tabela temporaria: total de ICMS por CFOP de ST, agregado por nota
SELECT N.SQNFOUTRA, NRCFOP,
        SUM(CASE WHEN VLICMS IS NULL THEN 0 ELSE VLICMS END) AS VLICMS,
        SUM(CASE WHEN VLICMSST IS NULL THEN 0 ELSE VLICMSST END) AS VLICMSST
INTO #126_C190_ST
FROM DBO.EFD_C190 N
INNER JOIN BI_DW_CORPORATIVO_FISC.DBO.TBLEG_CFOP O
        ON N.SQCFOP = O.SQCFOP AND NRCFOP IN
(1403,1407,1409,1411,1415,2403,2407,2409,2411,2415)
GROUP BY N.SQNFOUTRA, NRCFOP
HAVING SUM(CASE WHEN VLICMS IS NULL THEN 0 ELSE VLICMS END) > 0;

-- ... a procedure encadeia mais de dez tabelas #, cada uma consumindo o resultado
da anterior:
--      #126_PROP_ENTRADA -> casa NF-e propria com o registro C100 da EFD
--      #126_POCORRENCIA -> detecta lancamento duplicado
--      #126_ITEMP_ST -> junta o item da NF-e ao CFOP de ST
--      #126_ITEMP_ST_REF -> busca a nota original referenciada (caso de devolucao)
--      #126_ITEMP_ST_EMIT -> junta dados do emitente
--      #126_ITEMP_ST_DEST -> junta dados do destinatario
--      (o mesmo fluxo se repete em paralelo para notas de terceiros:
#126_TERC_ENTRADA, #126_ITEMP_ST ...)

-- 3) materializacao final: da ultima tabela temporaria para tabela permanente
DROP TABLE IF EXISTS
FISC.TB_126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA_01;

SELECT 'TABELA 01' AS IDENTIFICADOR_TABELA,
CONCAT(CONVERT(INTEGER,MONTH(DTINICIAL)) , '/', YEAR(DTINICIAL)) AS
REG00_PERIODO_DECLARACAO,
SUM(CASE WHEN ET.VLICMSC190_CFOP IS NULL THEN 0 ELSE ET.VLICMSC190_CFOP END) AS
TOTAL_C190_VL_ICMS
INTO FISC.TB_126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA_01
FROM #126_C190_ST_ITEM ET
GROUP BY YEAR(DTINICIAL), CONVERT(INTEGER,MONTH(DTINICIAL));

```



```
-- (o mesmo padrao SELECT INTO se repete para as tabelas _02, _03 e _04, em
granularidade crescente)

END

IF @RELATORIO IS NULL
BEGIN
    PRINT 'O TUTORIAL DESTA CONSULTA ENCONTRA-SE NO ROTEIRO OPERACIONAL DO BDFISC.
FAVOR, CONSULTA-LO.';
    -- devolve ate 4 result sets em sequencia, um por tabela _01 a _04,
    -- cada um filtrado por @ANO / @PERIODO_DECLARACAO
END;

END
GO
```

Visao geral, elemento por elemento:

1. **Tres parametros, todos opcionais (= NULL) -- @ANO e @PERIODO_DECLARACAO** filtram o periodo de apuracao; @RELATORIO continua sendo o "modo" (recalcular x so ler), igual ao estudo de caso 2.
2. **Validacao de parametro antes de executar a logica principal -- se @ANO e @PERIODO_DECLARACAO** foram informados e sao incompativeis entre si, a procedure devolve um **SELECT** de erro e nem chega a montar as tabelas temporarias. E um padrao de "guarda de entrada" (**input guard**) que vale a pena copiar em qualquer procedure parametrizada.
3. **Catorze DROP TABLE IF EXISTS #..., um por tabela temporaria --** feitos todos no comeco da procedure, antes de qualquer logica. Garante que, se a sessao ja tiver essas tabelas de uma execucao anterior (por exemplo, interrompida por erro), a nova execucao comeca limpa.
4. **Cadeia de tabelas temporarias --** cada #tabela novo consome o resultado da anterior (via **INNER JOIN**), num estilo parecido com um pipeline de ETL: agrega -> casa com outra origem -> filtra duplicidade -> enriquece com mais colunas -> materializa. Quebrar um calculo grande em varias tabelas # pequenas (em vez de uma unica consulta gigante) deixa cada etapa testavel isoladamente (basta rodar so ate aquele **SELECT INTO** e inspecionar a tabela #) e mais facil de dar manutencao.
5. **Materializacao final em tabela permanente --** assim como no estudo de caso 2, o resultado que precisa sobreviver ao fim da procedure (para o modo @RELATORIO IS NULL funcionar depois) vai para **FISC.TB_126_..._01..04**, tabelas sem #.
6. **Ate 4 result sets numa unica chamada --** quando @RELATORIO IS NULL, a procedure pode devolver quatro **SELECTs** em sequencia (um por tabela de

granularidade). No SSMS, cada um aparece como uma aba de resultado separada.

Exemplos de execucao com parametros

```
-- Consulta do resultado final (modo leitura)
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]
    @ANO = 2025,
    @PERIODO_DECLARACAO = '05/2025';

-- Recalculo do conjunto e materializacao das tabelas finais
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]
    @ANO = 2025,
    @PERIODO_DECLARACAO = '05/2025',
    @RELATORIO = 99999;
```

A primeira execucao e a que o analista usa para visualizar o resultado. A segunda e a operacao pesada: valida parametros, monta cadeias de **#temp**, gera as tabelas finais em **FISC.TB_126_..._01** a **_04** e depois encerra. Esse padrao mostra bem o papel da procedure como motor do processo.

Sugestao de reescrita usando visao

A versao em view pode ser feita como uma visao de “resultado ja pronto” e a filtragem por ano/periodo fica no **SELECT** da consulta, fora da definicao da view:

```
CREATE VIEW fisc.vw_entradas_credito_c190_cfop_st AS
SELECT
    IDENTIFICADOR_TABELA,
    REG0_PERIODO_DECLARACAO,
    TOTAL_C190_VL_ICMS,
    YEAR(DTINICIAL) AS ANO,
    MONTH(DTINICIAL) AS MES
FROM FISC.TB_126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA_01;
GO

SELECT *
FROM fisc.vw_entradas_credito_c190_cfop_st
WHERE ANO = 2025
      AND MES = 5;
```

Nesta reformulacao, a view substitui a tabela materializada somente na etapa final de consulta. Ela nao elimina a necessidade da procedure no processo de calculo, porque

a procedure continua sendo responsavel por montar o resultado e por controlar a validacao do periodo.

13. Boas praticas observadas no banco

1. Sempre **SET NOCOUNT ON** logo no inicio do corpo.
 2. Usar **DROP TABLE IF EXISTS** antes de recriar qualquer tabela (# ou permanente) para tornar a procedure segura para reexecucao.
 3. Ter um parametro "modo de operacao" (**@RELATORIO**) quando a procedure tanto recalcula quanto so consulta um resultado ja gerado -- evita duas procedures quase identicas.
 4. Validar combinacoes de parametros incompativeis no comeco da procedure, antes de qualquer processamento pesado.
 5. Preferir **#temp** para calculo intermediario e tabela permanente so para o resultado que precisa persistir.
 6. Tratar **NULL** explicitamente com **CASE WHEN ... IS NULL THEN 0/''** antes de gravar o resultado final -- evita que um **NULL** de origem vire ambiguidade no relatorio.
-

14. Erros comuns

1. **Esquecer SET NOCOUNT ON** -- nao quebra a procedure, mas polui a saida com mensagens de contagem de linha, o que atrapalha ferramentas que leem result set por posicao.
2. **Reexecutar SELECT ... INTO #tabela sem DROP TABLE IF EXISTS antes** -- gera **Msg 2714: There is already an object named '#tabela' in the database** na segunda execucao da mesma sessao.
3. **Confundir parametro com variavel DECLARED** -- parametro e definido no cabecalho e recebe valor na chamada (**EXEC ... @p = valor**); variavel **DECLARED** so existe dentro do corpo e nao pode ser passada de fora.
4. **Esperar que uma tabela #temp sobreviva a outra sessao** -- tabela # e visivel so na sessao que a criou; abrir uma nova janela de query no SSMS e nao encontrar a tabela # que "existia" na janela anterior e comportamento esperado, nao bug.

5. **Depender de tabela/banco vinculado sem tratar a ausencia dele** -- as tres procedures deste estudo de caso dependem de `BI_DW_CORPORATIVO_FISC` (banco corporativo); rodar sem esse link falha com `Msg 208: Nome de objeto ... invalido`. Em ambiente de treino, isso e esperado (ver os roteiros de auditoria linkados nesta apostila para consultas alternativas autocontidas).

15. Roteiro de laboratorio guiado

Este roteiro foi pensado para ser executado em sequênciã, no SSMS, como um laboratório prático. A ideia e que o aluno veja a teoria funcionando em passos pequenos, com observacao de resultado e reflexao sobre o comportamento de cada objeto.

Objetivo do laboratório

- reconhecer a diferença entre rotina operacional, regra reutilizavel e reação automática;
- identificar quando a procedure gera resultado final, quando a function entra em consulta e quando o trigger reage ao dado;
- praticar a passagem de parametros e a escolha entre tabela temporaria, tabela permanente e view;
- entender o papel de `SET NOCOUNT ON`, `DROP TABLE IF EXISTS` e `@RELATORIO` em procedures reais do banco fiscal.

Passo 1 — Preparação e inspeção do ambiente

O que fazer

```
USE 933000081200000173202662_20260127_104141;  
GO
```

```
SELECT name AS procedure_nome  
FROM sys.procedures  
WHERE name LIKE '%PR%'  
ORDER BY name;
```

O que observar

- A lista mostra as procedures do banco e ajuda a localizar o padrão de nomes (`PR_`, `100_PR_...`, `126_PR_...`).
- Isso confirma que o ambiente é orientado a rotina operacional e a relatórios de auditoria.

Pergunta de reflexão

- Por que o padrão de nomenclatura `_PR_` é importante para identificar procedimentos de processo e não de regra de cálculo?
-

Passo 2 — Verificar a estrutura dos parâmetros de uma procedure real

O que fazer

```
SELECT
    p.name AS parametro,
    TYPE_NAME(p.user_type_id) AS tipo,
    p.is_output,
    p.has_default_value,
    p.default_value
FROM sys.parameters AS p
WHERE OBJECT_NAME(p.object_id) =
    '126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA'
ORDER BY p.parameter_id;
```

O que observar

- Os parâmetros têm valores padrão `NULL`.
- A procedure é parametrizada e aceita diferentes modos de execução.
- Não há parâmetro `OUTPUT` no cenário real observado.

Pergunta de reflexão

- Qual a diferença entre receber um parâmetro para filtrar dados e receber um parâmetro para controlar o modo da rotina (`@RELATORIO`)?
-

Passo 3 — Executar a procedure em modo de leitura

O que fazer

```
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS];
```

O que observar

- A procedure devolve o painel de consulta final, sem recalculando a estrutura.
- Ela atua como uma visualização do estado já materializado.

Pergunta de reflexão

- O que acontece se as tabelas de apoio ainda não existirem? O que o padrão `IF EXISTS (...)` está tentando proteger?
-

Passo 4 — Executar a procedure em modo de recalculo

O que fazer

```
EXEC dbo.[100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS]  
@RELATORIO = 99999;
```

O que observar

- O corpo da procedure monta a tabela permanente de resumo.
- O resultado não é exibido imediatamente, mas a tabela é atualizada.

Pergunta de reflexão

- Por que esse código precisa primeiro gerar a tabela e só depois ler o resultado? O que isso diz sobre a natureza da rotina?
-

Passo 5 — Comparar o comportamento das duas execuções

O que fazer

```
SELECT NUMERO_CONSULTA,  
       DESCRICAO_CONSULTA,  
       INFOR_DADOS  
FROM FISC.TB_100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS_01  
ORDER BY NUMERO_CONSULTA;
```

O que observar

- Depois do recalcule, a tabela de painel possui dados consolidados.
- A chamada sem parâmetro apenas lê esse estado já materializado.

Conclusão

- A procedure possui duas faces: modo operacional e modo consulta.
 - Isso é um padrão muito comum em painéis fiscais e relatórios de auditoria.
-

Passo 6 — Testar a procedure com filtros de período

O que fazer

```
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]  
      @ANO = 2025,  
      @PERIODO_DECLARACAO = '05/2025';
```

O que observar

- O resultado final é devolvido em um ou mais **result sets**.
- A procedure toma o período e monta a análise fiscal correspondente.

Pergunta de reflexão

- O que a procedure está fazendo antes de devolver o resultado? O que a cadeia de tabelas temporárias representa?

Passo 7 — Recalcular a rotina complexa com parâmetros

O que fazer

```
EXEC dbo.[126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA]
    @ANO = 2025,
    @PERIODO_DECLARACAO = '05/2025',
    @RELATORIO = 99999;
```

O que observar

- O SQL Server executa várias etapas intermediárias antes de materializar tabelas finais.
- Há uso de `#temp` para etapas de cálculo e tabelas permanentes apenas para o resultado final.

Pergunta de reflexão

- Por que não gravar tudo de uma vez em tabela permanente desde o início? Qual é a vantagem de quebrar a rotina em etapas?

Passo 8 — Validar o uso de `DROP TABLE IF EXISTS`

O que fazer

```
DROP TABLE IF EXISTS #temp_teste;

SELECT 1 AS id, 'teste' AS valor
INTO #temp_teste;

SELECT * FROM #temp_teste;
```

O que observar

- A segunda execução do mesmo bloco, sem o `DROP TABLE IF EXISTS`, falha.

- Isso mostra por que as procedures colocam **DROP TABLE IF EXISTS** antes de recriar tabelas temporárias.

Pergunta de reflexão

- Em uma rotina fiscal executada várias vezes na mesma sessão do SSMS, qual é a vantagem de tornar a procedure reexecutável?
-

Passo 9 — Reconhecer a diferença entre rotina e consulta

O que fazer

```
SELECT
    p.name AS procedure_nome,
    p.object_id,
    TYPE_NAME(p.user_type_id) AS tipo_parametro,
    p.is_output
FROM sys.parameters AS p
WHERE OBJECT_NAME(p.object_id) = '100_PR_SUMARIO_DADOS_CONSULTAS_INCONSISTENCIAS';
```

O que observar

- A procedure usa parâmetros simples e não usa **OUTPUT**.
- O que a rotina entrega é um conjunto de resultados, não uma variável de negócio única.

Conclusão

- O SQL Server trata procedimento e consulta de modo diferente: a procedure faz fluxo e a consulta lê o dado final.
-

Passo 10 — Exercício prático de criação

O que fazer

Crie uma procedure simples para contar notas por UF, com o seguinte comportamento:

```
CREATE PROCEDURE dbo.pr_treino_total_nfe
    @sguf CHAR(2) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        n.id_emitente,
        c.uf,
        COUNT(*) AS total_notas
    FROM dbo.nfe AS n
    INNER JOIN dbo.contribuinte AS c
        ON c.id_contribuinte = n.id_emitente
    WHERE (@sguf IS NULL OR c.uf = @sguf)
    GROUP BY n.id_emitente, c.uf
    ORDER BY c.uf, n.id_emitente;
END;
GO
```

Execuções sugeridas

```
EXEC dbo.pr_treino_total_nfe;
EXEC dbo.pr_treino_total_nfe @sguf = 'PB';
```

O que observar

- Quando @sguf é NULL, a rotina mostra todas as UFs.
- Quando @sguf é informado, a rotina filtra pela UF.

Pergunta de reflexão

- Esse é um caso típico de procedure como rotina de consulta ou de processamento? Qual é o padrão de saída?

Passo 11 — Transformar a rotina em view para comparação

O que fazer

```
CREATE VIEW dbo.vw_treino_total_nfe AS
SELECT
    n.id_emitente,
    c.uf,
```

```

COUNT(*) AS total_notas
FROM dbo.nfe AS n
INNER JOIN dbo.contribuinte AS c
    ON c.id_contribuinte = n.id_emitente
GROUP BY n.id_emitente, c.uf;
GO

SELECT *
FROM dbo.vw_treino_total_nfe
WHERE uf = 'PB';

```

O que observar

- A view permite consulta direta e simples.
- Mas ela não substitui a procedure quando o objetivo é processar, filtrar por parâmetro, gerar financeiro de auditoria ou materializar um processamento em etapas.

Conclusão final do laboratório

- Procedure: executa fluxo, manipula dados e gera resultado;
- Function: responde lógica dentro de consulta;
- Trigger: reage automaticamente a evento de DML;
- View: expõe resultado já pronto, sem lógica operacional.

16. Exercícios propostos

1. Reescreva `PR_CADASTRO_CFOP` trocando a tabela temporaria `#CADCFOF` por uma variavel de tabela `@CADCFOF`. Aponte por escrito uma limitacao real dessa troca (dica: `SELECT ... INTO` nao funciona com `@tabela`).
2. A procedure `100_PR_SUMARIO...` usa duas variaveis `DECLARED` (`@QTDE_REG_TB1`, `@QTDE_REG_TB2`) para os casos em que uma consulta de auditoria tem duas tabelas de apoio (`_01` e `_02`, como no codigo 115). Explique por que usar `SUM` das duas variaveis, em vez de comparar cada uma isoladamente, muda o resultado logico do teste = 0.
3. Crie uma versao simplificada de `126_PR_ENTRADAS_CREDITO_C190_CFOP_SUBST_TRIBUT_MERCADORIAS_REVENDA` que funcione **sem** depender de `BI_DW_CORPORATIVO_FISC`, usando so `dbo.NFE` e `dbo.ITEM_NFE` (o codigo CFOP ja vem literal em `ITEM_NFE.cdcfop`, sem precisar de tabela de legislacao). Adicione um parametro `@RELATORIO OUTPUT` do

tipo `INT` que devolva a quantidade de itens suspeitos encontrados, sem usar result set para esse numero.

4. Adicione, na sua procedure do exercicio 3, o padrao `DROP TABLE IF EXISTS` antes de cada tabela temporaria criada, e justifique em uma frase por que isso importa mesmo numa procedure de uso didatico.